

Efficient Generation of Test Patterns Using Boolean Difference

Tracy Larrabee
Computer Science Department
Stanford University
Stanford, CA 94305

Abstract

Most automatic test pattern generation systems for combinational circuits generate a test for a given fault by directly searching a data structure representing the circuit to be tested. This paper describes a new system that divides the problem into two parts: First, it constructs a formula expressing the Boolean difference between the unfaulted and faulted circuits. Second, it applies a Boolean satisfiability algorithm to the resulting formula. The new system can incorporate any of the heuristics used by structural search techniques. It is not only quite general, but is able to test or prove untestable every fault in the popular Brglez-Fujiwara[1] test benchmark.

1 Introduction

A combinational circuit can be tested for the presence of a single stuck-at fault by applying a set of inputs (a test pattern) that excite a verifiable output response in that circuit. Given a circuit, the goal of an automatic test pattern generation (ATPG) system is not only to generate a set of test patterns that will detect every possible single stuck-at fault in the circuit, but also to identify all untestable faults in the circuit.

The time needed to generate one test pattern that will detect the presence of a single stuck-at fault is, in the worst case, exponential in the size of the circuit under test [4]. Many test generation systems have been designed and implemented [8,5,3,9] with the idea of avoiding those worst cases—of keeping the time needed to generate a test, in the expected case, within the realm of computational possibility. These systems all perform their search for a solution in a topological manner.

This paper describes a new approach to algorithmic generation of test patterns for combinational circuits. The method consists of two parts: First, given a circuit and a fault site in that circuit, construct a formula expressing the *Boolean difference* of a circuit with respect to a given fault. Second, apply a *Boolean satisfiability* algorithm to the resulting formula. Since the Boolean difference formula defines the complete set of tests capable of distinguishing

between the faulted and unfaulted circuits, satisfying this formula will give us a set of inputs that detect the fault.

The Boolean difference of a circuit and a fault has long been considered theoretically important [10,7], but the tedious nature of the algebraic manipulations involved in solving the Boolean difference led to its disfavor as a practical tool for test pattern generation. Fortunately, the Boolean difference need not be solved to be useful for generating tests: *satisfying* the formula is rewarding. Our system correctly tests or proves untestable every fault in the Brglez-Fujiwara benchmark system.

1.1 The Approach

Our system consists of a wirelist translator, followed by pseudo-random test pattern generation, followed by algorithmic test pattern generation.

During the first phase of test pattern generation, pseudo-random patterns are produced 32 at a time and simulated using a parallel pattern, single fault propagation (PPSFP) system modeled after the one reported by Waicukauski et al [11]. In this way we generate patterns for the 80% to 99% of the faults that are easily tested. When one complete PPSFP pass produces fewer than a predetermined number of patterns (currently two), the second phase, algorithmic pattern generation, begins. In the second stage we concentrate on raising the percentage of faults covered or proven uncoverable to 100%.

During the algorithmic pattern generation phase, each pattern generated is simulated (using a simple single pattern, single fault propagation simulator) so that any faults detected by the new pattern may be removed from the fault list. If too much time passes without generating a test pattern, the fault is abandoned. The performance of the complete system is reported in Section 5.

2 Extracting the Formula

A circuit is represented as a directed acyclic graph (a *dag*) with the sources of the graph being the outputs of the circuit and the sinks being the inputs of the circuit. By walking the graph starting at any output, each of the nodes that can af-

fect the value of that output are reached. Each node of the dag, a gate or a fanout point, is tagged with the logic formula, in 3-element conjunctive normal form, or 3CNF (also known as product of sums form). The formula associated with a logic element is a characteristic formula that is true if and only if the truth values assigned the variables representing the wires connected to the gate are consistent with the truth table for the element. For example, the formula for the AND gate shown in Figure 1 is $(\bar{D} + A) \cdot (\bar{D} + B) \cdot (D + \bar{A} + \bar{B})$. This formula is true if the variables A , B , and D take on values consistent with the formula $D = A \cdot B$. For comparison, the disjunctive normal form (or sum of products) version of the same formula is $A \cdot B \cdot D + \bar{A} \cdot B \cdot \bar{D} + A \cdot \bar{B} \cdot \bar{D} + \bar{A} \cdot \bar{B} \cdot D$.

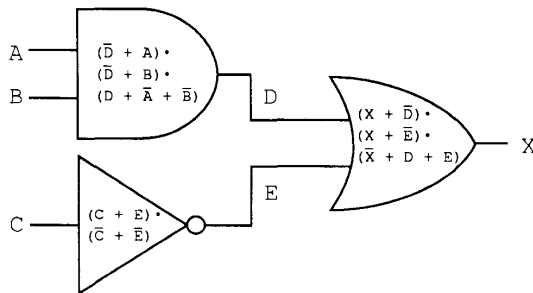


Figure 1: A Circuit

A formula describing the value of one of the circuit outputs in terms of its inputs can be obtained by walking the graph and taking the conjunction of all of the formulas for the visited nodes. Since variables are used to represent every wire of the circuit, the desired formula can be obtained in time and space that is linear in the size of the circuit. The formula for the output of the circuit in Figure 1 is $(X + \bar{D}) \cdot (X + \bar{E}) \cdot (\bar{X} + D + E) \cdot (\bar{D} + A) \cdot (\bar{D} + B) \cdot (D + \bar{A} + \bar{B}) \cdot (C + E) \cdot (\bar{C} + \bar{E})$.

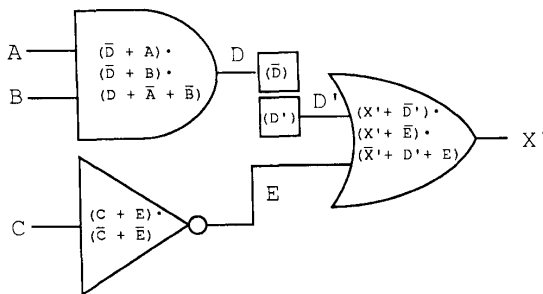


Figure 2: A Faulted Circuit

A faulted circuit is represented by a copy of its associated unfaulted circuit with renamed variables. Because the unfaulted and faulted circuits will have identical behavior except in those nodes that are affected by the fault, only the variables that are associated with wires that lie on a path between the fault site and a primary output need to

be renamed. Figure 2 shows a faulted circuit corresponding to the unfaulted circuit in Figure 1 with line D stuck-at 1. The formula for the output of the circuit in Figure 2 is $(X' + \bar{D}') \cdot (X' + \bar{E}) \cdot (\bar{X}' + D' + E) \cdot (D') \cdot (C + E) \cdot (\bar{C} + \bar{E})$.

The Boolean difference of a circuit with respect to a particular fault is defined as the XOR of the output of the unfaulted circuit and its associated faulted circuit output. The formula for the Boolean difference is obtained by walking the unfaulted circuit and the faulted circuit (as just described), and taking the conjunction of their formulas together with the formula for the XOR of the unfaulted output variable and the faulted output variable. The formula $BD = X \oplus X'$ is translated to $(BD = V_1 + V_2) \cdot (V_1 = X \cdot \bar{X}') \cdot (V_2 = \bar{X} \cdot X')$ where V_1 and V_2 are automatically generated new variables.

The formula for the Boolean difference resulting from the XOR of the output of the unfaulted circuit of Figure 1 and the faulted circuit of Figure 2 is $(X + \bar{D}) \cdot (X + \bar{E}) \cdot (\bar{X} + D + E) \cdot (\bar{D} + A) \cdot (\bar{D} + B) \cdot (D + \bar{A} + \bar{B}) \cdot (C + E) \cdot (\bar{C} + \bar{E})$ (the clauses contributed by the unfaulted circuit) $\cdot (X' + \bar{D}') \cdot (X' + \bar{E}) \cdot (\bar{X}' + D' + E) \cdot (D')$ (the clauses contributed by the faulted circuit) $\cdot (X + V_1) \cdot (X' + V_1) \cdot (V_1 + X + \bar{X}') \cdot (X + V_2) \cdot (\bar{X}' + V_2) \cdot (V_2 + \bar{X} + X') \cdot (\bar{B}\bar{D} + V_1 + V_2) \cdot (\bar{V}_1 + BD) \cdot (\bar{V}_2 + BD)$, (the clauses contributed by $BD = X \oplus X'$).

Generating a test pattern for the given fault is now a matter of satisfying this 3CNF Formula. If the formula cannot be satisfied, the fault is undetectable.

3 Satisfying the Formula

The problem of satisfying a 3CNF formula, *3SAT*, can be viewed as searching a binary tree. Each node of the tree corresponds to a variable in the formula to be satisfied, and the two branches of a node correspond to the two possible Boolean assignments to the variable associated with that node. A path from the root node to any other tree node is *consistent* with the formula if the partial binding associated with that path causes no 3CNF clause of the formula to evaluate to false (in the sense that all its literals evaluate to false). In order to satisfy the formula, a consistent path from the root node to any leaf node must be found.

The 3SAT problem was one of the first to be proven NP-complete [2]. This means that we have transformed one potentially exponential problem into another. However, the class of formulas generated by combinational circuits is an interesting sub-class of all 3CNF formulas, and we can use this fact to try to avoid the worst case behavior of 3SAT: At least two thirds of the clauses generated for the Boolean difference of a combinational circuit have only two disjuncts (are in 2CNF). This is true because each two-input unate gate contributes two binary (2CNF) clauses and one ternary clause. Unate gates with more than two inputs contribute more than two thirds binary clauses, and fanout points, buffers, and inverters contribute only binary clauses. In practice we have found that 80% to 90% of the clauses are in 2CNF. The problem of satisfying a 2CNF formula, 2SAT, is satisfiable in time linear in the number of clauses plus

the number of variables [2]. We may have an exponential number of 2SAT solutions, but we can use information from the ternary clauses to guide the iteration through the 2SAT assignments.

3.1 2SAT

We use a well-known algorithm for satisfying a 2CNF formula. The first step is to construct an *implication graph*. Each 2CNF clause $(X+Y)$ can be viewed as two implications: $\bar{X} \Rightarrow Y$ and $\bar{Y} \Rightarrow X$. The implication graph for a 2CNF formula shows all of the constraints imposed by 2CNF clauses on the logic values of the variables involved.

More formally, for each variable X occurring in the 2CNF clauses, there are two vertices in the graph, labeled X and $\bar{X}$. For every 2CNF clause $(X+Y)$ there are two edges in the graph: one from $\bar{X}$ to Y , and one from $\bar{Y}$ to X . The edge represents the logical implication between the two literals. We can now bind logic values to the variables in the graph. Any assignment is legal as long as it does not cause a node labelled true to precede (or imply) a node labelled false. Before we label the graph, we can simplify it by reducing *strongly connected components* to single nodes. A strongly connected component is a maximal set of nodes in a graph such that every node in the set is reachable from every other node in the set.

A strongly connected component represents a set of variables that are in an equivalence class. If any equivalence class contains both a literal and its negation, the formula is unsatisfiable. After each strongly connected component is reduced to a single node, the graph will not contain any cycles. Now we can find a binding for the 2CNF formula by visiting the vertices in any topological order. We choose a topological order that maximizes the number of variables in ternary clauses that are bound to false and thus narrowed to 2CNF or unary clauses.

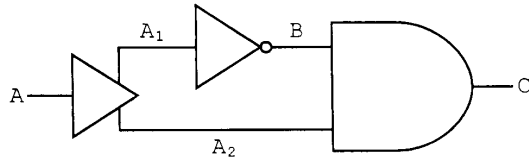


Figure 3: Another Circuit

As an example of how 2SAT works, consider the small circuit in Figure 3; imagine that we wish to iterate through all possible bindings to the variables A, A_1, A_2, B , and C . The formula for C is $(\bar{A} + A_1) \cdot (A + \bar{A}_1) \cdot (\bar{A} + A_2) \cdot (A + \bar{A}_2) \cdot (\bar{A}_1 + B) \cdot (\bar{A}_1 + \bar{B}) \cdot (\bar{C} + A_2) \cdot (\bar{C} + B) \cdot (\bar{A}_2 + \bar{B} + C)$. The implication graph of the 2CNF portion of this formula is shown in Figure 4. After the

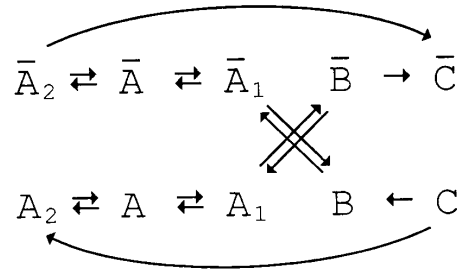


Figure 4: The Implication Graph

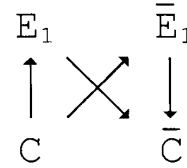


Figure 5: The Reduced Implication Graph

strongly connected components of this graph are reduced to unit nodes, the resulting graph is shown in Figure 5. The final graph clearly shows that $C \Rightarrow \bar{C}$, and therefore C must be bound to false. Given this restriction, only one node in the graph remains, and it can assume either Boolean value and remain consistent with the ternary clause.

3.2 Iterating through 2SAT Bindings

We've just described a method for constructing a satisfying assignment for the 2CNF portion of the formula by assigning values to the literals so that no node labelled true has a directed path to a node labelled false. Clearly there are many such assignments. We would like to iterate through these assignments in an order that maximizes our chances of quickly discovering a 2SAT solution that can be extended to a satisfying assignment for the entire 3CNF formula.

We begin our iteration by ordering the variables by a metric that combines the number of constraints the variable places on the other variables within the implication graph and the number of times the variable appears in the ternary clauses. This defines a total order on the 2SAT solutions: One assignment precedes another if the n -bit binary number representing the values of the variables (in the previously fixed order) precedes the n -bit binary number for the other assignment. We consider the 2SAT solutions in this order, using standard search techniques to attempt to extend each 2CNF solution to a solution for the entire formula.

There is an intuitive rationale behind the metrics used to heuristically order the search: If a partial binding is going to lead to a contradiction, we want to discover this as quickly as possible. By beginning our construction of a 2SAT solution with those variables with the largest number of constraints on other variables, we are more likely to discover any conflicts early in the construction.

4 Minimizing the Search Tree

The algorithm we have described is complete: If a solution exists, the system will eventually find it. However, we can speed up the satisfier tremendously by making sure that it does not explore parts of the search tree that are known to contain no solutions. The search tree may be reduced both by subtracting clauses from the formula and by adding clauses to the formula.

4.1 Removing Clauses

We can remove a variable from the formula (along with all the clauses containing the variable) if we are guaranteed that removing the variable will not cause a satisfiable formula to appear to be an unsatisfiable one. We can use the *determines* relation presented below to identify and remove extraneous variables from the formula.

We say that variable V determines variable W if both assignments to V cause W to appear in the formula only negated or only unnegated. In this case, we may remove all clauses containing W from the formula and postpone the assignment of W until after the final assignment of V has been made.

This technique can be used to mimic the behavior of the FAN algorithm [3], which stops its backtrace operation at *head lines*, wires guaranteed not to be involved in reconvergent fanout loops.

As an example, in the Boolean difference formula presented for the circuit in Figure 1, E determines C but C does not determine E . In fact, every variable in the formula but BD is determined by some other variable. Since the circuit from which we produced the formula is completely fanout free, it is not surprising that a satisfying binding can be found with no search.

A more interesting example appears in Figure 6. The characteristic formula for G would normally consist of 13 clauses, but the removal of all clauses containing variables A , B , and C will leave only 8 clauses in the remaining formula because F determines A and E determines B and C .

Unfortunately, our technique as stated will not remove as many variables as may be safely removed from the formula: the technique will not remove variables corresponding to input wires for XOR gates not involved in reconvergent fanout loops. We are working on a modification that will correctly deal with XOR gates.

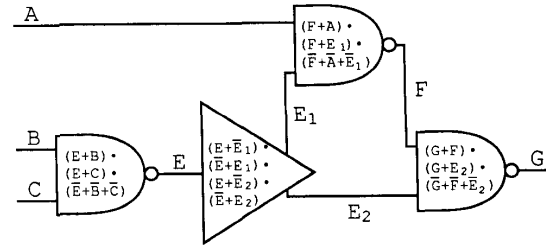


Figure 6: The Formula for G can be reduced to $(E + \overline{E_1}) \cdot (\overline{E} + E_1) \cdot (E + \overline{E_2}) \cdot (\overline{E} + E_2) \cdot (F + E_1) \cdot (G + E_2) \cdot (G + C) \cdot (\overline{G} + \overline{E_2} + \overline{C})$

4.2 Adding Clauses

We can take the basic formula to be satisfied and add clauses that explicitly state information that can be derived by the satisfier, but may only be derived after a certain amount of case-splitting. The simplest example of such redundant information is the value of the unfaulted wire with the fault. For example, the formula for the fault shown in Figure 2 contains the unary clause (D') . The satisfier can derive that the variable (D) must take on the value false in order for the XOR of the faulted and unfaulted circuits to be equal to true, but can add that information explicitly, by adding the clause $\overline{D}$ to the formula. Adding this kind of derivable information can speed up the satisfier by an order of magnitude. The effects of the added clauses mentioned here are fully described in another paper [6], but three different kinds of redundant information to be added are described in this section.

4.2.1 Active Clauses

One important class of redundant information is a set of clauses that speed fault propagation. If a fault is detectable, there must be at least one path from the fault site to a primary output such that every wire on that path has different unfaulted and faulted values. There may be more than one such path, but we only need to find one.

For every wire that lies on a path between the fault site and a primary output, we allocate an *active* variable. If the active variable for a wire is true, then the faulted value of the wire differs from the unfaulted value. For example, for the circuit in Figure 2 we allocate the variables Act_D and Act_X , and add the clauses $(\overline{Act_D} + D + D')$, $(\overline{Act_D} + \overline{D} + \overline{D}')$, $(\overline{Act_X} + X + X')$, and $(\overline{Act_X} + \overline{X} + \overline{X}')$ to the formula to be satisfied. We also add clauses that state that if an input to a single-output node is active, the output is active; if an input to a multi-output gate is active, one of the outputs is active; and the fault site is always active. Figure 7 and Figure 8 show the clauses that are added for circuit elements with fanin and for circuit elements with fanout.

It is important to notice that the active implication clauses (as shown in Figures 7 and 8) for the circuit as a whole can be used to determine all of the unique sensi-

zation points (points of total reconvergence) in a circuit.

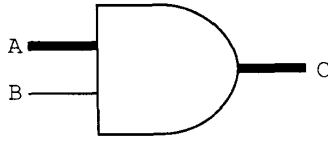


Figure 7: If A is active, C must be active: $(\overline{Act_A} + Act_C)$

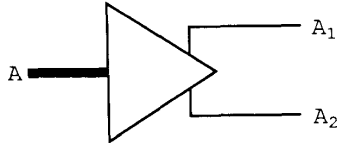


Figure 8: If A is active, either A_1 or A_2 must be active: $(\overline{Act_A} + Act_{A_1} + Act_{A_2})$

4.2.2 Critical Clauses

If a node is on the active path, we know that node must propagate the fault (discrepancy). This means that the non-active inputs to the node must have taken on critical values that allow the fault to be propagated. Non-active inputs to XOR and XNOR gates on the active path must not have a discrepancy. Non-active inputs to gates implementing monotonic functions must either have a discrepancy identical to that of the active input, or have no discrepancy and assume a static critical value (AND and NAND gates require static critical values of true, and OR and NOR gates require static critical values of false). Figure 9 shows two legal critical assignments for a 4-input AND gate (the active path is shown by a bold line), and Figure 10 shows an illegal assignment for the same gate. In these illustrations, the notation 0/1 means that the wire takes on the value 0 (false) in the unfaulted circuit, and 1 (true) in the faulted circuit. For example, the OR gate in Figure 2 will cause the clause $(\overline{Act_D} + \overline{E})$ to be added to the formula to be satisfied.

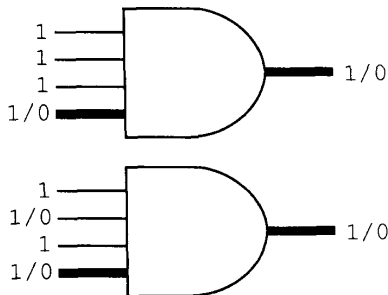


Figure 9: Legal Critical Assignments

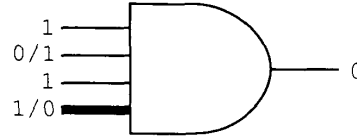


Figure 10: An Illegal Critical Assignment

4.2.3 Non-local Implications

As pointed out by Schulz et al [9], it is possible to explicitly derive non-local implications by examining the reconvergent fanout in a circuit. Figure 11 presents a circuit that demonstrates this idea: If wire B has the value true, wire F has the value true. This means that we know that if wire F has the value false, wire B has the value false. This implication can be discovered by performing a structural analysis of the circuit, or it may be found by analyzing the formula representing the circuit.

We can list all of the non-local implications of a given variable assignment by binding the variable and then noting the implications that use a ternary clause. Any implication that involves a ternary clause must come from reconvergent fanout. If variable B non-locally implies variable F , the clause $(\overline{B} + F)$ may be added to the formula to be satisfied.

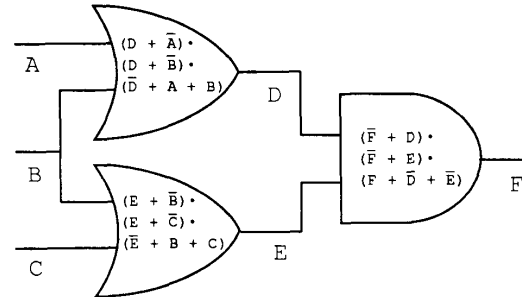


Figure 11: Non-local Implications: Add $(\overline{B} + F)$

5 Experimental Results

We produced test sets for ten sample circuits collected by Franc Brglez [1] and described by a paper in the Proceedings of the 1985 ISCAS Conference.

The test generation was run on a Titan, an experimental RISC machine developed at the Digital Equipment Corporation Western Research Laboratory. A Titan is about 10 times faster than a VAX-11/780. The implementation is written in Modula-2.

Table 1 shows the total time spent for each individual circuit. Tables 2 and 3 report on the faults tested, and proved untestable by the different phases of the program (every fault was either covered or proved untestable). In each case,

Circuit	Time in Seconds	
	Random	Algorithmic
C0432	.9	9.0
C0499	1.0	8.2
C0880	2.1	38.1
C1355	9.6	16.2
C1908	10.5	109.3
C2670	5.4	350.0
C3540	38.9	271.3
C5315	8.7	92.1
C6288	130.8	63.1
C7552	24.2	570.4

Table 1: Timing

the largest percentage of the faults were covered by the patterns generated by the first phase of the system. (For the C6288 circuit, every pattern was produced by the pseudo-random phase, and the algorithmic phase was restricted to proving that all remaining faults were uncoverable.) Table 4 shows the number of patterns produced by each phase.

We believe that these experimental results show that the generation of test patterns by extracting a formula and then satisfying it is a general and practical alternative to traditional structural search methods.

Circuit	Percentage of Faults		
	Covered	Redundant	Aborted
C0432	99.03	0.97	0.0
C0499	98.64	1.36	0.0
C0880	100.00	0.0	0.0
C1355	99.45	0.55	0.0
C1908	99.48	0.52	0.0
C2670	94.84	5.16	0.0
C3540	95.89	4.11	0.0
C5315	98.76	1.24	0.0
C6288	99.55	0.45	0.0
C7552	98.15	1.85	0.0

Table 2: Coverage

6 Acknowledgements

This paper grew out of work begun with Greg Nelson of Digital Equipment Corporation's Systems Research Center. Guidance was provided by the author's advisor, Professor John Hennessy. The wirelists for the Brglez circuits were provided to the author by Jon Udell, who was then a student in Stanford's Center for Reliable Computing.

The author was supported by a Digital Equipment Corporation Student Fellowship during the course of this research. In addition to monetary support, Digital, through its Western Research Lab, provided access to many powerful

Circuit	Covered by			Proved Untestable
	Original	Random	Algorithmic	
C0432	411	394	13	4
C0499	588	555	25	8
C0880	748	710	38	0
C1355	1444	1389	47	8
C1908	1740	1452	279	9
C2670	2250	1810	324	116
C3540	3136	2906	101	129
C5315	4761	4665	37	59
C6288	7616	7582	0	34
C7552	7069	6487	451	131

Table 3: Number of Faults

Circuit	Number of Patterns	
	Random	Algorithmic
C0432	70	7
C0499	53	19
C0880	93	21
C1355	90	18
C1908	64	110
C2670	94	79
C3540	197	71
C5315	191	27
C6288	47	0
C7552	211	137

Table 4: Number of Patterns

machines and enthusiastic lab members. Special thanks are due to Mary Jo Doherty and David Boggs for proof-reading this manuscript.

References

- [1] F. Brglez, H. Fujiwara, "A Neutral Netlist of 10 Combinatorial Benchmark Circuits and a Target Translator in FORTRAN", Special Session on Recent Algorithms for Gate-Level ATPG with Fault Simulation and Their Performance Assessment, *International Symposium on Circuits and Systems* June 1985.
- [2] S. A. Cook, "The Complexity of Theorem Proving Procedures," *Proceedings of the Third Annual ACM Symposium of Theory of Computing*, 1971 151-158.
- [3] H. Fujiwara and T. Shimono, "On the Acceleration of Test Generation Algorithms," *IEEE Transactions on Computers*, vol.C-30, December 1983, 1137-1144.
- [4] H. Fujiwara and S. Toida, "The Complexity of Fault Detection Problems for Combinational

- Logic Circuits," *IEEE Transactions on Computers*, vol. C-31, June 1982, 555-560.
- [5] P. Goel, "An Implicit Enumeration Algorithm to Generate Tests for Combinational Logic Circuits," *IEEE Transactions on Computers*, vol. C-30, March 1981, 215-222.
 - [6] T. Larrabee, "A Framework for Evaluating Test Pattern Generation Strategies," *Proceedings of the International Conference on Computer Design*, 1989.
 - [7] E. J. McCluskey, *Logic Design Principles*, Prentice-Hall Publishing, 1986.
 - [8] J. P. Roth, "Diagnosis of Automata Failures: A Calculus and a Method," *IBM Journal of Research and Development*, vol. 10, July, 1966, 278-291.
 - [9] M. H. Schulz, E. Trischler, and T.M. Sarfert, "SOCRATES: A Highly Efficient Automatic Test Pattern Generation System," *IEEE Transactions on CAD*, January 1988, 126-137.
 - [10] F. F. Sellers, M. Y. Hsiao, And L. W. Bearnson, "Analyzing errors with the Boolean difference," *IEEE Transactions on Computers*, vol. C-17, July 1968, 676-683.
 - [11] J. A. Waicukauski, E. B. Eichelberger, D. O. Forlenza, E. Lindbloom, T. McCarthy, "Fault Simulation for Structured VLSI," vol. VI, December 1985, 20-32.